

Kubernetes - First Project

Activate local website via Docker and Kubernetes

Our goal is to create a static website which the server is supported by Kubernetes.

We have some components: 0) We have a Makefile with commands - it's not part of the real build. 1) website directory - written with HTML, CSS and JavaScript. 2) nginx.conf file - We use nginx (inside container) for our web server and this is the configuration file for the server. 3) Dockerfile - This is the image file to create the container where the nginx web server lives. 4) kind_config.yaml - Configures the Kubernetes Kind cluster, including port mappings and a local container registry for hosting images. 5) kind_configurationmap.yaml - Defines a Kubernetes ConfigMap that stores metadata about the local registry, making it accessible to other Kubernetes components.

If we wanted to use our website without Kubernetes all we need to do is: 1) `docker build -t explorecalifornia.com .` - Build the image file that contains a nginx web server, copying the website file for the web server and config nginx file and informs Docker that the container will listen on port 80 for HTTP traffic. 2) `docker run -p 5000:80 -d --name explorecalifornia.com --rm explorecalifornia.com` - Create and run the container of the image file. Also maps port 80 inside the container (where the Nginx server is listening) to port 5000 on your local machine. We can access the web server by visiting `http://localhost:5000` in your browser.

But we want Kubernetes so this is the commands we use by order:

- 1) `curl -o ./kind https://github.com/kubernetes-sigs/kind/releases/download/v0.11.1/kind-darwin-amd64` - Install kind. It is a tool for running Kubernetes clusters locally, using Docker containers as the nodes in the cluster.
- 2) `brew install kubectl` - Install kubectl. kubectl is the command-line tool used to interact with Kubernetes clusters. It allows you to deploy, manage, and monitor applications, as well as inspect and control the Kubernetes cluster and its resources.
- 3) `kind create cluster --image=kindest/node:v1.21.12 --name explorecalifornia.com --config ./kind_config.yaml` - Create a cluster with kind.
- 4) `docker run -d -p 5000:5000 --name local-registry --restart=always registry:2` - Create and run a container which will be used as local-registry for docker images.
- 5) `docker network connect kind local-registry` - Connects the local-registry container to the kind Docker network. Pay attention we use another container here with an image file called `registry:2` for image files local-registry.
- 6) `docker build -t explorecalifornia.com . && docker tag explorecalifornia.com localhost:5000/explorecalifornia.com && docker push localhost:5000/explorecalifornia.com` - Only now when we have docker registry we build the image and push it

to the registry.

- 7) `kubectl apply -f ./kind_configmap.yaml` - Create a resource for called ConfigMap for the applications.
- 8) `kubectl create deployment --dry-run=client --image localhost:5000/explorecalifornia.com explorcalifornia.com --output=yaml > deployment.yaml` - It will create a deployment.yaml file with some properties for a Deployment app.
- 9) `kubectl apply -f deployment.yaml` - It will create A deployment app in our Kuberenetes. Actual Pods will be created now. 9_extra) `kubectl port-forward deployment/explorecalifornia.com 8080:80` - Make a port mapping to enable send request from port 8080 in the local machine and get the result from port 80 of the container. It's only for checking - We will use another Service to do the port mapping.
- 10) `kubectl create service clusterip --dry-run=client --tcp=80:80 explorcalifornia.com --output=yaml > service.yaml` - Create a Cluster IP service to create IP address for communication with our website.
- 11) `kubectl apply -f service.yaml` - Create the service. We changed the yaml file so the name will be without a dot because of the DNS server error so the name was explorecalifornia-svc. 11_extra) `kubectl port-forward service/explorecalifornia-svc 8080:80`
- 12) `kubectl apply -f https://kind.sigs.k8s.io/examples/ingress/deploy-ingress-nginx.yaml` - Some kind of installation of Ingress NGINX.
- 13) `kubectl create ingress explorecaliforina.com --rule="explorecaliforina.com/=explorecali" --dry-run=client --output=yaml > ingress.yaml` - Creae yaml file for ingress.
- 14) `kubecte apply -f ingress.yaml` - Create ingress. We change the yaml to hold Prefix in the pathType field.
- 15) Add `127.0.0.1 explorecalifornia.com` to `/etc/hosts` via `sudo vim /etc/hosts`. The NGINX ingress controller matches on the "host" provided within the Ingress. Consequently, connecting from "localhost" would not be accepted by the Ingress.

Kubernetes ConfigMap

A Kubernetes ConfigMap is a resource that lets you store and manage configuration data for your applications. It allows you to separate application configuration from the application code, making it easier to update or reuse configurations without modifying the app.

A higher-level abstraction provided by Kubernetes for application configuration.

Used to store configuration data for applications running in the cluster.

Its purpose is to externalize application-specific configuration (e.g., environment variables, file-like configurations) to make the app more flexible and easier to manage.

It simplifies working with configuration data by integrating seamlessly into Pods,

Deployments, etc., via environment variables or volumes.

You can interact with ConfigMaps using kubectl or APIs, and they are scoped to namespaces.

etcd

Acts as the core data store for Kubernetes itself.

Used to store cluster state, metadata, and configuration for Kubernetes components (e.g., Pods, Deployments, Nodes, Secrets).

It is essential for Kubernetes' control plane to function.

A low-level distributed key-value database.

Used internally by Kubernetes to maintain the desired and actual state of the cluster (e.g., objects like ConfigMaps, Secrets, Nodes).

Used by Kubernetes internally; not directly accessed by developers or applications. Typically managed by the Kubernetes control plane and administrators.

Cluster IP Service

Cluster IP service create a virtual IP address that maps all the pods that are running inside of our deployment it also creates a DNS inside the cluster

Ingress

It's a reverse proxy that enables access into other kubernetes resources. Like aws load-balancer. Ingress gets a series of routines rules.

Kubernetes - Helm

Helm Let's you install apps onto Kubernetes like you install apps on computer. It's a package manager.

It packages all the application's manifest into something called Helm Chart.

Helm consists of: 1) Helm metadata, 2) Chart values, 3) Chart templates.

Create Helm Chart: 1) **brew install helm** 2) Create chart.yaml file manually. This is an app. 3) Create values.yaml file manually. This file will store the chart values. 3_optional) **helm show all** - gives us all the data about the chart. 4) **mkdir templates** - We need such directory to store the templates. 5) Create new deployment.yaml, ingress.yaml and service.yaml with variables instead of hard coded names inside templates directory. 5_optiona) **helm template** - render templates and see they are good. 6) **helm install explore-california-website ./chart** - Create helm chart.

Kubernetes - Real Live working website in a kubernetes cluster

We use AWS EKS.

AWS provide us ALB ingress controller for the load balancer instead of the one of ingress.

We will need to create the cluster in AWS - we have the `create_cluster.sh` script for that. Keep in mind for config values to connect to aws.

Useful commands: - `aws eks update-kubeconfig --name explorecalifornia.com`
- `aws ecr describe-repositories` - `aws ecr get-login-password` - `aws ecr get-login-password` is what you would run to get the password for your Docker registry within ECS. Remember, 'AWS' is always the user when using ECS. - `docker login "$registry" --username AWS --password "$password"` - `docker tag explorecalifornia.com:latest "${registry}:latest"` - `docker push "${registry}:latest"` - `kubectl create secret docker-registry explore-california --docker-server=$register --docker-username=AWS --docker-password=$password`

You need to modify your Service so that it creates a "NodePort" type Service if it is using a "ClusterIP" type. You will also need to modify your Ingress to add annotations that tell Kubernetes to use this ingress controller. The resources are located in `AES_EKS`.

You do need to push your image into ECR, you also need to ensure that the Secret containing your Docker registry credentials are present.